

VidyaTrack Frontend Architecture: Long-Form Engineering Documentation

1. Document Purpose

This document explains how the VidyaTrack frontend is designed, how major systems fit together, how data moves through the application, and how engineers should extend the codebase safely.

This is written for:

- senior frontend engineers
- full-stack engineers contributing to the frontend
- engineering managers reviewing architecture
- future maintainers onboarding into the system

The intent is the same standard expected in large engineering organizations:

- clear boundaries
- explicit ownership
- predictable extension paths
- low-risk change management

2. Product Context

VidyaTrack is a role-based school operations platform. The frontend supports multiple user surfaces:

- Authentication
- Platform administration
- Management setup and administration
- Principal oversight
- Teacher execution workflows
- Student detail and profile experiences

The application must support school-scoped multi-tenancy while keeping role experiences isolated.

That means the architecture must solve for:

- strict role separation
- school-aware data fetching
- controlled route access
- reusable shared infrastructure
- maintainable feature growth

3. Core Architectural Principles

The frontend currently follows these principles.

3.1 Thin Route Shells

Route files should mostly:

- gate loading/error states
- compose subcomponents
- hand off orchestration to feature hooks

Route files should not:

- own large transformation logic
- embed repeated mutation behavior
- manage complex derived state inline

3.2 Feature-Local Ownership

Feature-specific logic stays inside feature folders until reuse becomes real.

This avoids the common failure mode of premature “shared” abstractions that become hard to discover and harder to change.

3.3 Shared Infrastructure Only When Reuse Is Proven

Top-level shared modules exist for code that is demonstrably reused:

- query keys
- school context helpers
- API helpers
- shared shell UI
- common concurrency utility

3.4 Data and UI Separation

The architecture intentionally separates:

- data loading and side effects
- business rules
- formatting/transformation
- rendering

3.5 Preserve User Behavior First

Refactoring in this codebase is not treated as redesign.

The primary rule is:

- same UI
- same routing
- same interaction pattern
- same toasts and edge cases
- same or better performance

4. Tech Stack

4.1 Runtime

- React
- TypeScript
- Vite
- React Router
- TanStack React Query
- Zustand
- React Hook Form
- Axios

4.2 Styling

- Tailwind CSS utility-driven styling
- local UI primitives under `src/components/ui`

4.3 Icons

- `lucide-react`
- `react-icons`

5. High-Level Application Layers

The app can be understood as three layers.

5.1 App Assembly Layer

Location:

- `src/app`
- `src/layouts`

Responsibilities:

- boot sequence
- global providers
- router composition
- role layout selection
- error boundaries

5.2 Feature Layer

Location:

- `src/features/\*`

Responsibilities:

- user-visible product workflows
- feature-local UI
- orchestration hooks
- business rules
- feature-specific types and constants

5.3 Shared Infrastructure Layer

Location:

- `src/api`
- `src/hooks`
- `src/constants`
- `src/helpers`
- `src/components`

- ``src/store``
- ``src/utls``

Responsibilities:

- remote data access
- query identity
- school-scoped helpers
- cross-feature reusable UI
- app-wide state

6. Boot Sequence

Entry file:

- ``src/main.tsx``

Boot process:

1. CSS is loaded
2. ``AppProviders`` is mounted
3. in development, auth API helpers are exposed for console-level verification
4. the root is rendered in ``StrictMode``

Why this matters:

- all providers are centralized
- dev-only testing affordances do not leak into production UI behavior

7. Provider Model

Provider composition is centralized under the app layer so individual features do not need to re-declare global infrastructure.

Key provider responsibilities:

- Router provider
- Query client provider
- any future app-wide providers such as analytics or feature flags

This keeps provider registration stable and avoids provider drift across routes.

8. Routing Architecture

Root router:

- ``src/app/router.tsx``

Role route modules:

- ``src/app/routes/auth.routes.tsx``
- ``src/app/routes/platform.routes.tsx``
- ``src/app/routes/teacher.routes.tsx``
- ``src/app/routes/principal.routes.tsx``
- ``src/app/routes/management.routes.tsx``

Why modular routing is important

- role trees are readable in isolation
- permission wrappers stay near route definitions
- adding new routes no longer expands one giant central file
- onboarding becomes simpler because each role surface is its own route map

Role access model

Protected routes are wrapped by ``RoleGuard``.

Allowed mappings:

- ``super_admin`` -> platform routes
- ``management`` -> management routes
- ``principal`` -> principal routes
- ``teacher`` -> teacher routes

Auth routes remain unguarded but layout-scoped.

9. Layout Architecture

Layouts now follow the same decomposition principles as features.

9.1 App Shell

Primary orchestration:

- sidebar state

- breadcrumb derivation
- hard-stop state
- idle timeout behavior
- retry/logout/support actions

Internal structure:

- `src/layouts/appShell/hooks`
- `src/layouts/appShell/components`
- `src/layouts/appShell/helpers`
- `src/layouts/appShell/types`

9.2 Header

Split into:

- brand cluster
- actions cluster
- date formatting util

9.3 Sidebar

Split into:

- sidebar header
- account menu/footer
- helper mapping for labels/icons/classes

This separation means layout changes are smaller and less risky.

10. Feature Folder Standard

Every major feature now follows a normalized structure when complexity justifies it.

Typical layout:

- `components/`
- `hooks/`
- `helpers/`
- `utils/`
- `types/`
- `constants/`

Responsibilities

`components/`

- render-only
- props-driven
- minimal or no side effects

`hooks/`

- state orchestration
- query wiring
- mutation flows
- navigation side effects
- selection logic

`helpers/`

- pure domain rules
- validation
- derivation logic
- comparisons

`utils/`

- formatting
- display shaping
- serialization/deserialization

`types/`

- extracted local contracts
- view model types
- prop interfaces when reused

`constants/`

- static labels
- preset lists
- configuration-like values

11. Shared Query Strategy

Canonical query keys now live in:

- `src/constants/queryKeys.ts`

This was introduced because the previous codebase repeated ad hoc query key arrays throughout hooks and features.

Why centralized query keys matter

- prevents cache collisions
- prevents invalidation drift
- improves refactor safety
- makes cross-feature fetch behavior easier to reason about

Current key families

- academic setup
- classes
- sections
- subjects
- teachers
- principal teachers
- management principal
- management principal history
- teaching assignments
- teacher context
- teacher readiness
- teacher attendance section
- teacher self teaching assignments
- students
- principal students
- students by section
- detailed students by section
- student notes
- student profile
- attendance root
- attendance by section and date
- existing marks
- principal marks
- principal attendance

- parent message history
- homework history
- platform school dashboard
- platform school teachers/students/staff
- platform school dashboard fallback

12. School Context Strategy

This application is school-scoped.

School context is a first-class part of the architecture, not a convenience value.

Shared helpers:

- ``src/helpers/requireSchoolId.ts``
- ``src/api/helpers/schoolParams.ts``

``requireSchoolId``

Purpose:

- fail fast in mutations or flows that require an active school
- remove repeated guard logic from hooks

``schoolParams``

Purpose:

- standardize ``school_id`` request param shaping
- avoid repeated inline parameter objects across API modules

Engineering rule

If a request is school-scoped, use shared school helpers rather than hand-rolling local variants.

13. API Layer Design

API modules live under ``src/api``.

Responsibilities:

- endpoint selection

- request serialization
- params shaping
- response typing

Not allowed to own:

- route navigation
- page state
- UI messages
- mutation orchestration

Example domains

- auth
- attendance
- marks
- teachers
- students
- management principal
- teaching assignments
- schools

14. Shared Hook Design

Shared hooks in `src/hooks` now follow more consistent rules:

- school-aware keys
- explicit enabled conditions
- standardized invalidation
- extracted shared utilities for repeated patterns

Examples:

- `useClasses`
- `useSections`
- `useSubjects`
- `useStudents`
- `useTeachers`
- `useTeachingAssignments`
- `useManagementPrincipal`
- `useExistingMarks`

- `useAttendanceSubmit`
- `useMarksSubmit`

15. Batch Mutation Strategy

Marks and attendance both required bounded concurrency.

Shared utility:

- `src/utls/runWithConcurrency.ts`

Reason for extraction:

- duplicated utility logic existed in multiple hooks
- both workflows need the same batching semantics
- keeping it shared reduces silent divergence

Design principle:

- per-record record/upsert calls are separated from final workflow submission
- backend side effects happen only in the final submit phase

16. Role Surface: Platform

Platform is the super-admin surface.

Main areas:

- dashboard
- schools list
- school detail
- school creation wizard

Platform Create School

Now decomposed into:

- step header
- step content
- step alerts
- step footer
- reusable selection grid
- review cards

- step descriptions constant

This makes the onboarding wizard easier to extend without re-opening one monolithic component.

17. Role Surface: Management

Management owns institutional setup workflows.

Primary areas:

- dashboard
- principals
- teachers setup
- students setup
- students import
- subjects
- section/class setup
- subject assignment setup
- schools setup

Management now has clear feature-local folders for these flows, which improves maintainability and reduces page complexity.

18. Role Surface: Principal

Principal owns oversight and reporting.

Main areas:

- dashboard
- attendance history
- marks history
- communications
- student lookup
- teacher lookup

Principal Communications

This was one of the highest-complexity flows and is now split into:

- shared tab shell
- header
- toasts
- context card
- notes flow hook
- delivery flow hook
- helper derivations

19. Role Surface: Teacher

Teacher owns execution workflows.

Main areas:

- dashboard
- attendance entry
- marks entry
- communications
- notes
- student drill-down

Teacher and principal now share more of the communications rendering model while keeping orchestration role-specific.

Shared teacher top-bar UI lives at:

- ``src/components/teachers/TeacherFeatureTopBar.tsx``

20. Role Surface: Student

Student-related UI is currently profile- and list-oriented rather than a separate authenticated surface.

Main areas:

- students list
- student profile

Student profile now has:

- hook-based orchestration
- extracted helper logic

- extracted rendering cards
- filtered notes handling
- report-card generation flow

21. Communications Subsystem

The communications subsystem is one of the most reusable frontend designs in the repo.

Main flows:

- homework broadcast
- parent messages
- student notes

Current design:

- tab shell owns active panel selection
- composer cards own write paths
- timeline cards own read paths
- selector/search component is isolated
- orchestration hooks own state transitions and mutation behavior

Benefits:

- easier to explain
- easier to test
- easier to add role-specific variants without duplicating full panels

22. Marks Subsystem

Marks entry and marks history were refactored to preserve:

- hydration of existing marks
- max marks locking
- submit sequencing
- dashboard redirect payload behavior
- exam context reset behavior

Shared improvements:

- concurrency utility extraction
- query-key standardization

- better isolation of table and summary rendering

23. Attendance Subsystem

Attendance flows preserve:

- optimistic update behavior where already implemented
- date and section scoped querying
- final submission semantics
- dashboard redirect/toast handoff

Shared improvements:

- common school-scoped API param shaping
- common query-key strategy
- reusable batch execution utility

24. Principal and Management Lookup Flows

Student and teacher lookup flows were split into:

- filter/header/table components
- hook-driven orchestration
- role-aware navigation

These pages now follow the same structure as the more complex teacher workflows.

25. Error and Feedback Strategy

UI feedback relies on shared feedback components:

- ``LoadingState``
- ``ErrorState``
- ``EmptyState``
- ``AppErrorBoundary``

Pattern:

- route shell decides which state to render
- hooks expose stable booleans and data
- feature components stay free from large loading/error branching when possible

26. Form Strategy

React Hook Form is used where structured entry is needed.

Examples:

- management teacher onboarding
- school creation
- student creation/import-adjacent flows

Rule:

- form registration and submit logic stay in hooks
- pure field rendering stays in components

27. What Was Cleaned Up During Refactor

This repo-wide pass removed or normalized:

- repeated query key literals
- repeated `school\_id` param objects
- repeated missing school guards
- repeated batch concurrency utilities
- stale imports
- dead empty folders
- large route files mixing all concerns
- repeated local derivation code where reuse was real

28. Build Optimization

The last pass added safe manual chunk splitting in:

- `vite.config.ts`

Result:

- removed the earlier single large-bundle warning
- split runtime into meaningful vendor groups
- preserved runtime behavior and route behavior

Current chunk pattern includes:

- `react-vendor`
- `state`

- `ui-vendor`
- `vendor`
- app bundle

29. How to Extend the Codebase Correctly

When building a new feature:

1. create or update the route shell
2. create a feature-local hook for orchestration
3. split UI into small presentational units
4. move pure rules to `helpers`
5. move formatting to `utils`
6. define or reuse query keys
7. use school-scoped helpers if the data is tenant-aware

When extracting shared code:

- only promote it to top-level `src/\*` if at least two real features need it
- prefer discoverability over abstraction density

30. Explaining This Codebase to a New Engineer

The simplest mental model is:

- router chooses the role surface
- layouts define the shell
- page shells define what gets rendered
- hooks decide how the page behaves
- APIs fetch and mutate data
- query keys identify remote cache entries
- helpers encode business rules
- utils format data

31. Operational Standards for Contributors

Preferred engineering behavior:

- preserve behavior before improving structure
- keep components presentational
- keep side effects in hooks

- prefer pure extraction before optimization
- invalidate the smallest correct cache scope
- avoid introducing global shared buckets for feature-only code

32. Current Repository End State

After the refactor:

- the route layer is modular
- the feature layer is decomposed
- shared hook conventions are standardized
- school-scoped APIs are normalized
- frontend lint is clean
- frontend build is clean
- bundle splitting is improved

33. Recommended Next Steps

The main remaining work is not structural refactoring. It is optional quality investment:

- automated tests for critical hooks and route flows
- CI workflow for lint/build
- optional route-level lazy loading if product performance budgets demand it
- role-specific onboarding docs for junior engineers

34. Reference Files

Primary entry points and conventions:

- ``src/main.tsx``
- ``src/app/router.tsx``
- ``src/app/routes/*``
- ``src/layouts/*``
- ``src/store/auth.store.ts``
- ``src/constants/queryKeys.ts``
- ``src/helpers/requireSchoolId.ts``
- ``src/api/helpers/schoolParams.ts``
- ``src/utls/runWithConcurrency.ts``

35. Summary

The frontend is now organized for maintainability at scale:

- role-oriented route composition
- feature-oriented implementation
- clean separation of UI, logic, and data access
- standardized multi-tenant school handling
- predictable shared data conventions

That is the main architectural outcome of the refactor.